ASSIGNMENT- 11.3

Name:T.Harivarshith

HT.No: 2303A51351

Batch: 20

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Sample input

1. Implement the contact manager using arrays (lists).

2. Implement the same functionality using a linked list for dynamic memory allocation.

3. Implement the following operations in both approaches: o Add a contact o Search for a contact o Delete a contact

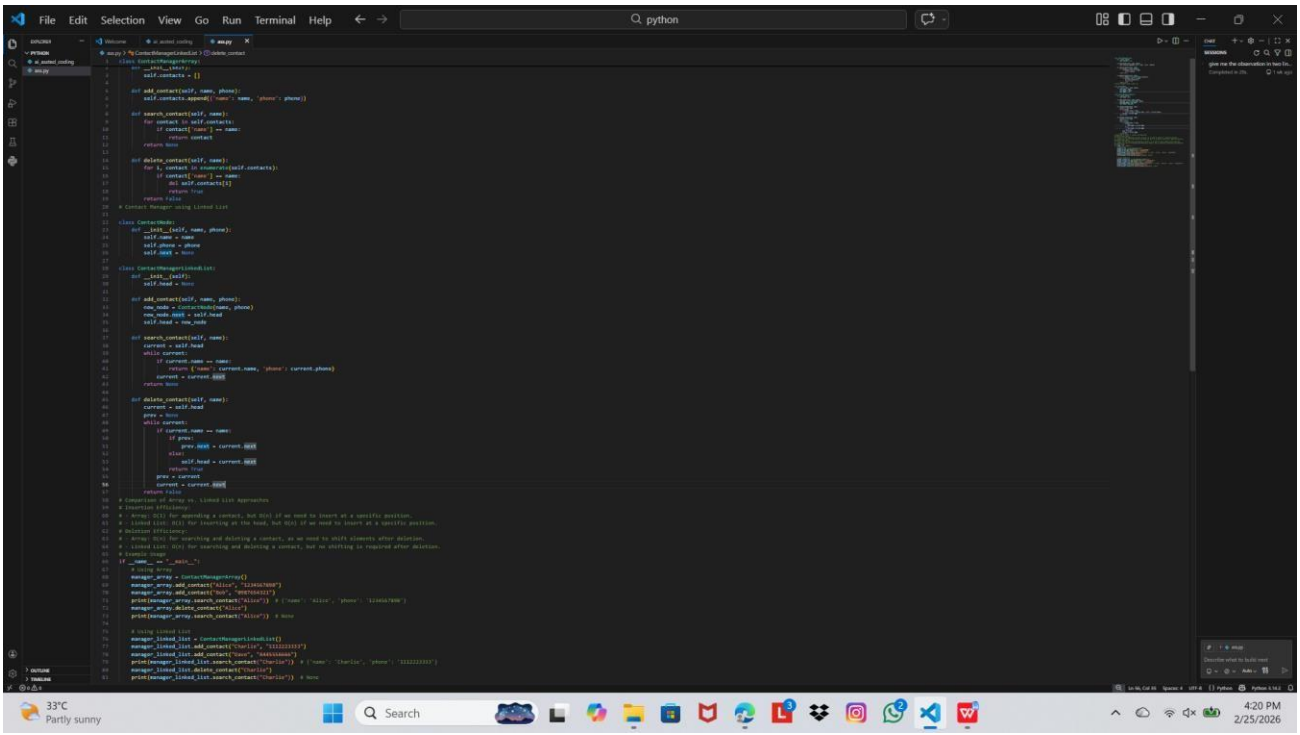
4. Use GitHub Copilot to assist in generating search and delete methods.

5. Compare array vs. linked list approaches with respect to:

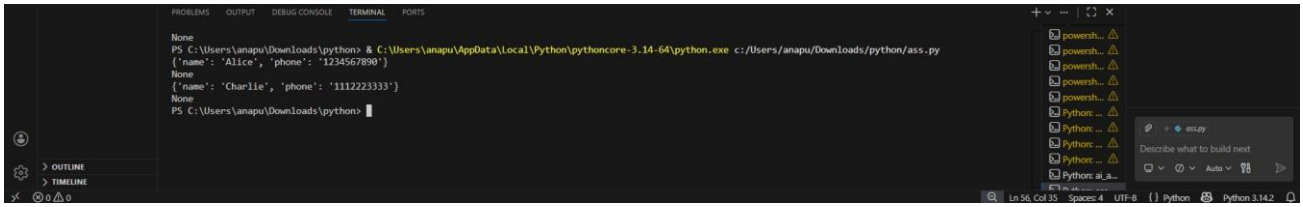
o Insertion efficiency o

Deletion efficiency

Code & probmt



Result:



Observation:

- The array-based contact manager allows simple and fast appending (O(1)) but requires shifting elements during deletion, making removals O(n).

- The linked list version enables efficient insertion at the head (O(1)) without shifting on deletion, though searching remains O(n) in both approaches.

Task 2: Library Book Search System (Queues & Priority Queues) Sample
Input:
The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.
Tasks
1. Implement a Queue (FIFO) to manage book requests.
2. Extend the system to a Priority Queue, prioritizing faculty requests.
3. Use GitHub Copilot to assist in generating:
o enqueue() method o
dequeue() method
4. Test the system with a mix of student and faculty requests.
Prompt &Code:

```python
class BookRequest:
    def __init__(self, requester_type, book_title):
        self.requester_type = requester_type  # 'student' or 'faculty'
        self.book_title = book_title
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        return None

    def is_empty(self):
        return len(self.items) == 0
class PriorityQueue:
    def __init__(self):
        self.faculty_queue = Queue()
        self.student_queue = Queue()

    def enqueue(self, request):
        if request.requester_type == 'faculty':
            self.faculty_queue.enqueue(request)
        else:
            self.student_queue.enqueue(request)

    def dequeue(self):
        if not self.faculty_queue.is_empty():
            return self.faculty_queue.dequeue()
        elif not self.student_queue.is_empty():
            return self.student_queue.dequeue()
        return None
# Example Usage
if __name__ == "__main__":
    priority_queue = PriorityQueue()
    priority_queue.enqueue(BookRequest('student', 'Book A'))
    priority_queue.enqueue(BookRequest('faculty', 'Book B'))
    priority_queue.enqueue(BookRequest('student', 'Book C'))

    print(priority_queue.dequeue().book_title)  # Book B (faculty request)
    print(priority_queue.dequeue().book_title)  # Book A (student request)
    print(priority_queue.dequeue().book_title)  # Book C (student request)
    print(priority_queue.dequeue())  # None (no more requests)
```

Result:

```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ass.py
Book B
Book A
Book C
None
PS C:\Users\anapu\Downloads\python>
```

Observation:

- The priority queue ensures faculty requests are always processed before student requests, regardless of arrival order.

Within each category (faculty or student), requests are handled in FIFO order, preserving fairness among the same requester type

Task 3: Emergency Help Desk (Stack Implementation)

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

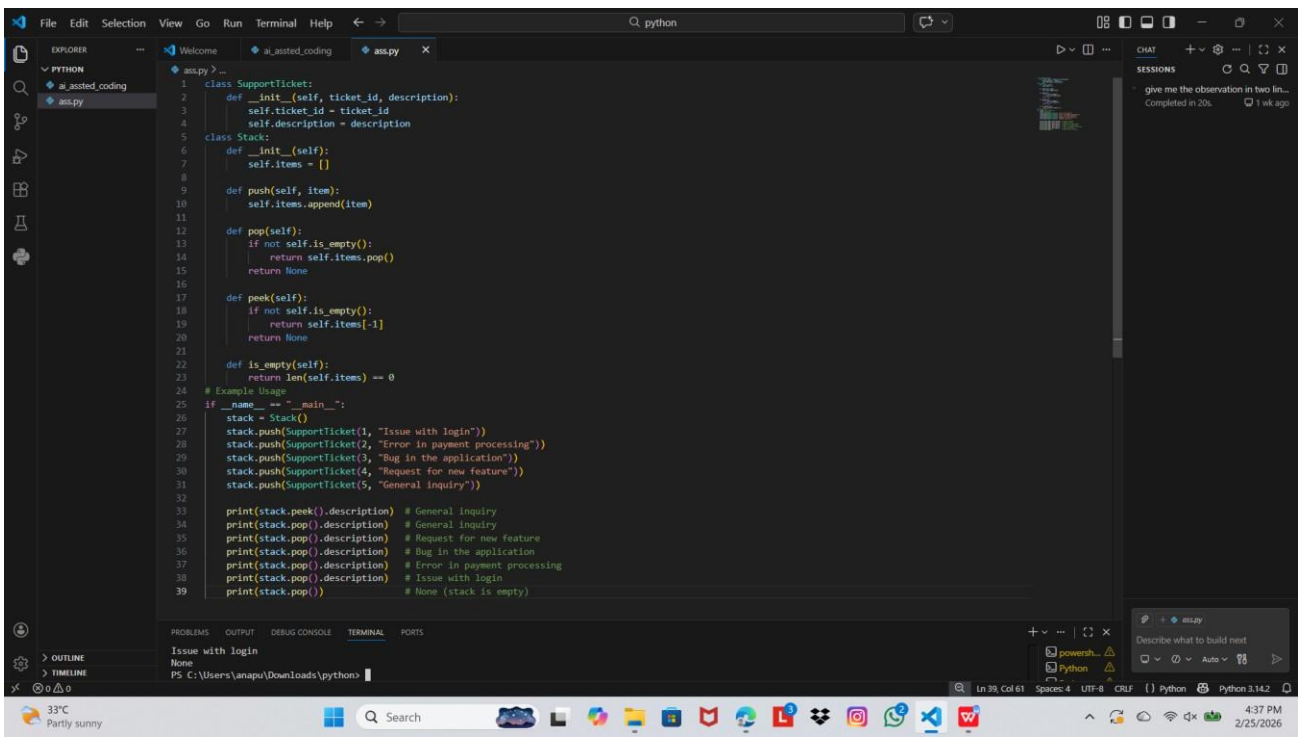
Tasks

1. Implement a Stack to manage support tickets.

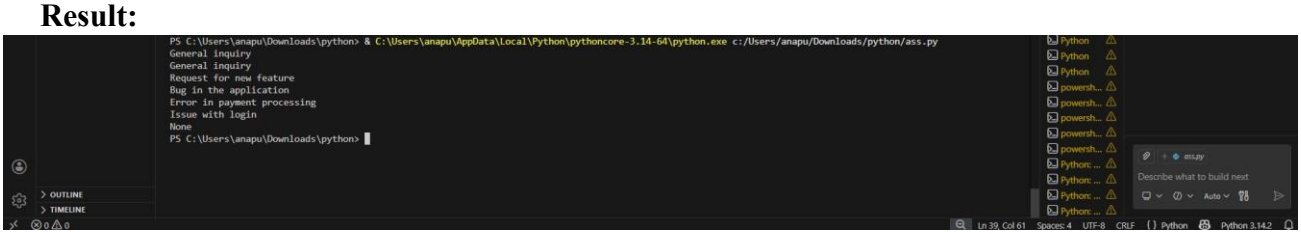
2. Provide the following operations: o push(ticket) o pop() o peek()

3. Simulate at least five tickets being raised and resolved.

4. Use GitHub Copilot to suggest additional stack operations such as: o Checking whether the stack is empty o Checking whether the stack is full (if applicable) Prompt & Code:



Result:



Observation:

The stack follows the LIFO (Last In, First Out) principle, so the most recently added support ticket is processed first.

The `peek()` method allows viewing the top ticket without removing it, and attempting to `pop()` from an empty stack safely returns `None`

Task 4: Hash Table

Objective

To implement a Hash Table and understand collision handling Sample

input:

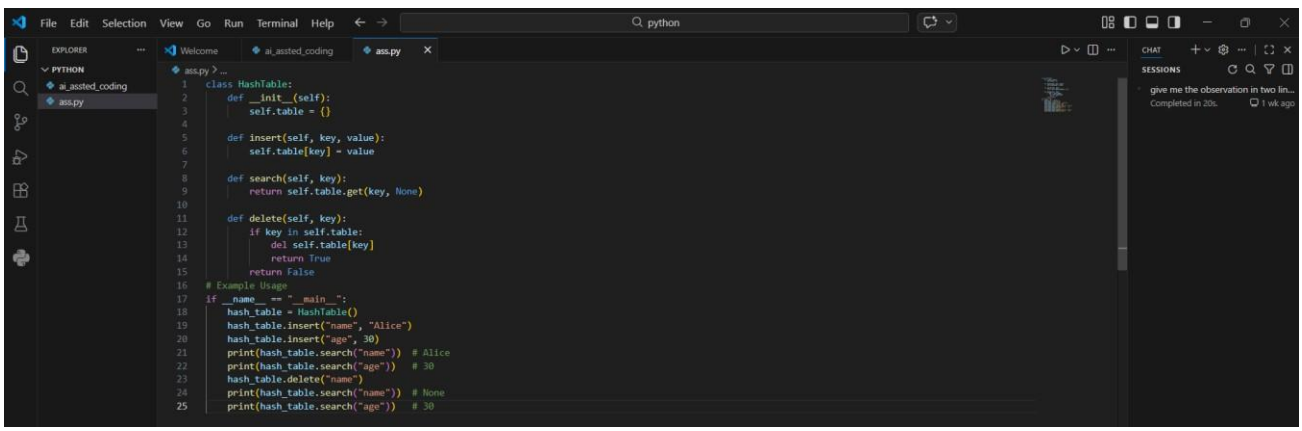
Task Description

Use AI to generate a hash table with:

- **Insert**
- **Search**
- **Delete Starter Code class HashTable:**

pass

Prompt &Code:



Result:

Observation

The hash table provides fast average-case time complexity of O(1) for insertion, search, and deletion operations using key-based access.

After deleting a key, searching for it returns `None`, while other existing keys remain unaffected in the table.

Task 5: Real-Time Application Challenge

Sample input:

Design a Campus Resource Management System with the following features:

- **Student Attendance Tracking**
- **Event Registration System**
- **Library Book Borrowing**
- **Bus Scheduling System**
- **Cafeteria Order Queue**

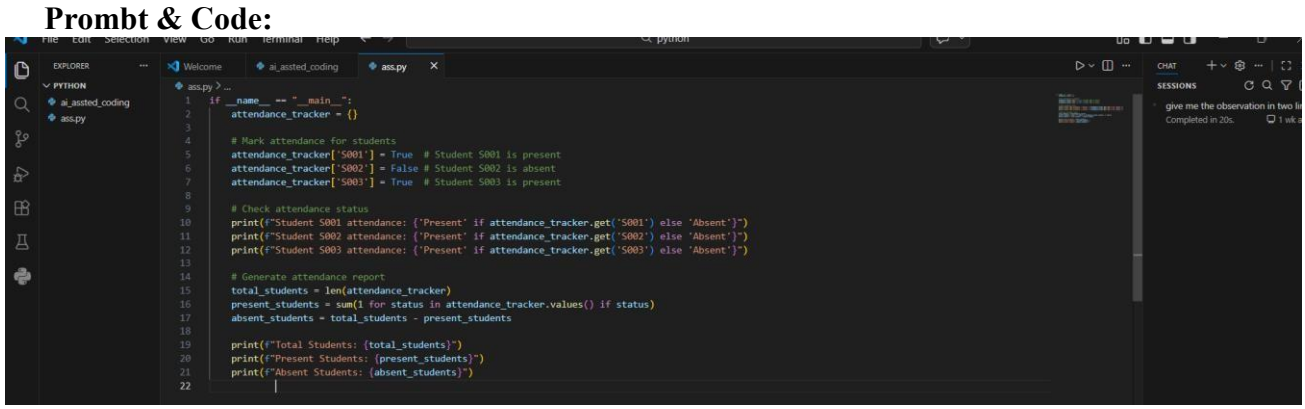
Student Tasks

1. Choose the most appropriate data structure for each feature.

2. Justify your choice in 2–3 sentences.

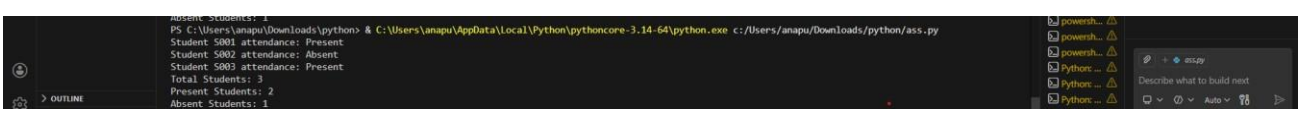
3. Implement one selected feature using AI-assisted code generation.

Prombt & Code:



```python
if __name__ == "__main__":
    attendance_tracker = {}

    # Mark attendance for students
    attendance_tracker['S001'] = True  # Student S001 is present
    attendance_tracker['S002'] = False # Student S002 is absent
    attendance_tracker['S003'] = True  # Student S003 is present

    # Check attendance status
    print(f"Student S001 attendance: {'Present' if attendance_tracker.get('S001') else 'Absent'}")
    print(f"Student S002 attendance: {'Present' if attendance_tracker.get('S002') else 'Absent'}")
    print(f"Student S003 attendance: {'Present' if attendance_tracker.get('S003') else 'Absent'}")

    # Generate attendance report
    total_students = len(attendance_tracker)
    present_students = sum(1 for status in attendance_tracker.values() if status)
    absent_students = total_students - present_students

    print(f"Total Students: {total_students}")
    print(f"Present Students: {present_students}")
    print(f"Absent Students: {absent_students}")
```

Result:



```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ass.py
Student S001 attendance: Present
Student S002 attendance: Absent
Student S003 attendance: Present
Total Students: 3
Present Students: 2
Absent Students: 1
```

Observation:

The dictionary efficiently tracks attendance using student IDs as keys, allowing quick O(1) access to each student's status.

The attendance report is generated by counting boolean values, making it simple to calculate total, present, and absent students dynamically.